Next, let's update the schema.sql file to add the columns for the new fields to the `book` table.

Listing 5.12 Adding columns for audit metadata to the `book` table

```
DROP TABLE IF EXISTS book;
CREATE TABLE book (
  id                  BIGSERIAL PRIMARY KEY NOT NULL,
  author              varchar(255) NOT NULL,
  isbn                varchar(255) UNIQUE NOT NULL,
  price               float8 NOT NULL,
  title               varchar(255) NOT NULL,
  created_date        timestamp NOT NULL,      ❶
  last_modified_date  timestamp NOT NULL,      ❷
  version             integer NOT NULL
);
```

❶ When the entity was created (stored as a timestamp)

❷ When the entity was last modified (stored as a timestamp)

I'll leave it to you to update the autotests accordingly, when necessary. You can also extend the tests in `BookJsonTests` to verify the serialization and deserialization of the new fields. As a reference, you can check Chapter05/05-intermediate/catalog-service in the code repository accompanying this book.

So far, you have got everything in place to map your Java objects to relational objects in a database, including audit metadata. You still need a way to access data from the database, though. That's the subject of the next section.

5.2.4 Data repositories with Spring Data

The *repository* pattern provides an abstraction for accessing data independently of its source. The `BookRepository` interface used by `BookService` is an example of a repository. The domain layer, which